

From Ephemeral Context to Project Truth: Reconciling State Across Interoperable Software Engineering Agents

Anonymous Author(s)

Abstract

As artificial intelligence software engineering agents become increasingly interoperable, a critical failure mode emerges at the boundary of cross-agent handoffs and multi-session persistence. Current systems treat agent-generated output as generic "memory" or "context," failing to distinguish between exploratory proposals, verified code implementations, and superseded decisions. This paper empirically investigates the required state, evidence, and governance primitives necessary to preserve continuity and accountability across heterogeneous agents (e.g., Claude Code, OpenCode). We propose the transition from passive agent memory to formal *Project Truth*—an authoritative, reconciled state model grounded in version control and test outcomes. Through a series of controlled experiments ([EMPIRICAL RESULTS PENDING: EXP-001 reference] and [EMPIRICAL RESULTS PENDING: EXP-002 reference]), we evaluate whether structured work-state transfer and persistent project intelligence significantly reduce rework, hallucination, and contradiction rates compared to raw transcript replay. Our conceptual framework defines the lifecycle of agentic claims from proposal to verification, demonstrating that effective multi-agent engineering requires reconciliation and commitment control rather than mere context retrieval. [EMPIRICAL RESULTS PENDING: Final conclusion summary pending empirical data.]

CCS Concepts

• Software and its engineering → Software development techniques; • Computing methodologies → Multi-agent systems.

Keywords

AI Agents, Software Engineering, Multi-Agent Systems, Context Transfer, Agent Interoperability, Project State

ACM Reference Format:

Anonymous Author(s). 2026. From Ephemeral Context to Project Truth: Reconciling State Across Interoperable Software Engineering Agents. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (August 2026), 4 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

The rapid integration of Large Language Models (LLMs) into the software engineering lifecycle has led to the proliferation of autonomous coding agents, such as Claude Code and OpenCode [2]. While these agents demonstrate high proficiency in isolated, single-session tasks, their practical utility diminishes when operating in complex, multi-day, or multi-developer environments. The primary failure mode in this paradigm is context amnesia: agents function as ephemeral silos, losing vital architectural rationale, constraints, and state transitions across session boundaries.

Current approaches attempt to resolve this discontinuity by treating project state as a retrieval problem, utilizing vector databases and semantic search (e.g., [5, 6]) to inject historical transcripts into an agent's context window. However, this "Project Memory" approach suffers from a fundamental flaw: it conflates unverified agent proposals with authoritative project decisions. Over time, memory stores accumulate noise, obsolete hypotheses, and discarded code paths, leading to cascading hallucinations and compounding rework.

We hypothesize that resolving multi-agent discontinuity requires a shift from passive *Memory* to formal *Project Truth*. To preserve accountability and continuity across agentic handoffs [1], systems must reconcile disparate claims against deterministic evidence, such as version control commits and test suite outcomes, establishing a governed lifecycle of commitments [8].

In this paper, we present the *Bridge* framework, an authoritative project truth and reconciliation substrate designed to facilitate interoperable software engineering agents. Through a sequence of controlled experiments ([EMPIRICAL RESULTS PENDING: EXP-001 and EXP-002 results]), we evaluate the impact of structured work-state transfer and persistent project intelligence on agent performance.

Our core contributions are:

- A formalization of the agent interoperability boundary, transitioning from context transfer to governed project truth.
- An empirical evaluation of structured work-state handoffs across heterogeneous agents ([EMPIRICAL RESULTS PENDING: reference to EXP-001 empirical claims]).
- A conceptual framework for state reconciliation and commitment control in multi-agent software engineering.

2 Problem Definition

The core problem in continuous, multi-agent software engineering is the degradation of shared state fidelity across the interoperability boundary. When work transitions between agents (e.g., from an analysis agent to an execution agent) or between sessions of the same agent, the transfer mechanism fundamentally shapes the likelihood of downstream success.

We define three primary dimensions of state degradation:

- (1) **The Token Inefficiency Problem:** Standard agent interfaces transmit state via raw conversational transcripts. As transcripts grow, they consume substantial token budgets and increase generation latency, while simultaneously diluting critical task constraints with intermediate conversational noise.
- (2) **The Rework Anti-Pattern:** Receiver agents, lacking explicit structured metadata regarding what has already been attempted or discarded, frequently re-explore identical failure paths. This results in circular debugging loops and elevated API costs.

- (3) **The Memory Contamination Vulnerability:** When systems attempt to persist state across sessions by indexing all conversational output (Phase 2: Project Memory), they invariably index hallucinations and superseded proposals. Without a formal reconciliation mechanism to invalidate obsolete claims, agents retrieve and act upon contradictory “facts.”

Fundamentally, current architectures lack a *governed state primitive* [3, 7] capable of differentiating between what an agent *said* and what the project *is*.

3 Background

3.1 The Evolution of Agent Interoperability

The development of AI-assisted software engineering has followed a recognizable architectural trajectory:

- (1) **Multi-Agent UIs:** Early approaches provided unified interfaces to interact with multiple models, acting as shallow routers without sharing persistent state.
- (2) **Context Transfer:** Emerging standards like the Agent-to-Agent (A2A) protocol [1] defined JSON-RPC mechanisms for agents to hand off task instructions, though often relying on unpruned transcripts.
- (3) **Project Memory:** Platforms (e.g., [5, 6]) introduced cross-session semantic storage, embedding and retrieving agent conversations to approximate long-term memory.

3.2 Knowledge Graphs and Code Intelligence

Recent frameworks have begun bridging the gap between natural language agent memory and deterministic code structure. Graph-based memories [4] attempt to construct entity relationships from code syntax trees (ASTs). The Model Context Protocol (MCP) [2] standardizes the extraction of local filesystem and tool context. While these systems provide robust ingestion of *facts* (e.g., “File X contains function Y”), they do not govern *intent* or *decisions* (e.g., “Function Y was deprecated because it causes memory leaks”).

3.3 Commitment Protocols

In distributed multi-agent systems literature, coordination is often modeled through *commitments* [8]. A commitment is a normative relationship between agents, establishing obligations and expectations. Applying this to LLM-driven software engineering requires mapping natural language proposals to verifiable codebase alterations, establishing a rigid chain of provenance from agent hypothesis to test-verified reality.

4 Research Questions

Our empirical evaluation is guided by the following overarching question: *When heterogeneous AI agents become interoperable, what persistent state, evidence, authority, and governance primitives are required to preserve continuity?*

We decompose this into three specific, empirically testable research questions:

- **RQ1 (Work-State Transfer Efficiency):** Does structured metadata transfer (the ‘ExperimentalWorkTransfer’ schema)

significantly reduce receiver agent rework, token consumption, and failure rates compared to raw transcript replay during a single-hop task handoff?

- **RQ2 (Project Intelligence Continuity):** Does maintaining a longitudinal, verified project intelligence layer across multiple sessions reduce the incidence of architectural contradictions and hallucinated constraints compared to fresh, memory-less sessions?
- **RQ3 (Reconciliation and Governance):** How does the enforcement of a strict commitment lifecycle (differentiating between *proposed*, *decided*, *implemented*, *verified*, and *superseded* states) impact the frequency of unauthorized or stale agent actions?

5 Experimental Methodology

To answer the research questions, we designed a reproducible empirical program executing isolated agentic software engineering tasks within a deterministic harness.

5.1 Environment and Control Mechanics

To prevent cross-trial contamination and ensure determinism:

- **Isolated Worktrees:** Every trial executes in a freshly cloned, ephemeral Git worktree initialized at a strict base commit hash.
- **Model Pinning:** The receiver agent (OpenCode) is strictly pinned to a specific, verified model endpoint ([**EMPIRICAL RESULTS PENDING: e.g., nvidia/nvidia/nemotron-3-super-120b-a12b**]) invoked via NDJSON streaming to prevent UI-layer non-determinism.
- **Metrics Collection:** The experimental harness automatically extracts and validates outcome metrics without LLM-based summary hallucination. Measurements include token consumption, exit codes, task duration, and automated test suite status.

5.2 EXP-001: Cross-Agent Work Transfer

EXP-001 isolates the single-hop transmission boundary. A primary analysis agent (Claude Code) initiates a task, and upon interruption or delegation, the state is transferred to a receiver agent (OpenCode) across three controlled conditions:

- **Condition A (Baseline):** The receiver agent receives the raw task prompt with zero prior context.
- **Condition B (Transcript Transfer):** The receiver agent receives the raw, unpruned conversation transcript of the primary agent, capped at 8KB.
- **Condition C (Structured Transfer):** The receiver agent receives a JSON-RPC compliant ExperimentalWorkTransfer payload containing distilled metadata, verified facts, and unresolved hypotheses extracted via the Bridge logic layer.

All conditions are evaluated against a standardized refactoring task.

5.3 EXP-002: Persistent Project Intelligence

[**EMPIRICAL RESULTS PENDING: Pending justification from EXP-001**]. Assuming structured transfer provides empirical gains, EXP-002 evaluates longitudinal persistence.

- **Condition A:** A sequence of three interconnected coding sessions executed by an agent with zero persistence between sessions.
- **Condition B:** The identical sequence executed with Bridge project intelligence injected as initial context in Sessions 2 and 3.

5.4 Metrics and Primary Outcomes

The primary outcomes measured across all experiments are correctness, automated test success rates, and successful task completion. Secondary outcomes include token consumption, execution duration, payload size, and the rate of manual human intervention required to resolve agent deadlocks.

6 Results

[EMPIRICAL RESULTS PENDING: This section is explicitly blocked pending the successful execution of the EXP-001 empirical pilot and subsequent replication runs. No results will be reported until verified machine-readable telemetry is captured from the experimental harness.]

6.1 EXP-001: Work-State Transfer Metrics

[EMPIRICAL RESULTS PENDING: Will contain Table 1: Individual Trial Results, reporting Correctness, Tests, Time, Rework, Intervention, Token Usage, Cost, and Payload Size for Conditions A, B, and C.]

6.2 EXP-002: Project Intelligence Continuity

[EMPIRICAL RESULTS PENDING: Will contain Table 2: Multi-Session Persistence Metrics, pending justification and execution.]

7 Analysis

[EMPIRICAL RESULTS PENDING: This section is explicitly blocked pending empirical data from EXP-001.]

7.1 Causal Mechanisms

[EMPIRICAL RESULTS PENDING: Ablation analysis of observed benefits (e.g., whether gains in Condition C are driven by token reduction, explicit negative constraints, or artifact grounding) will be conducted post-experiment.]

8 Conceptual Framework: Project Truth and Reconciliation

Based on the theoretical limitations of unpruned context windows and passive memory systems, we propose a governance framework for multi-agent software engineering. We hypothesize that long-term agent interoperability requires the management of state through a rigid reconciliation lifecycle.

8.1 The Deterministic Boundary

A fundamental principle of the Bridge architecture is the separation of deterministic facts from probabilistic inferences.

- **Deterministic Ground Truth:** Elements such as Git commit histories, Abstract Syntax Trees (ASTs), static typecheck results, and test suite exit codes represent absolute reality.
- **Probabilistic Claims:** Agent proposals, intent summaries, and architectural rationale represent unverified claims.

Project Truth is formed only when a probabilistic claim is reconciled against deterministic ground truth.

8.2 The State Lifecycle

We model agentic claims as transitioning through five formal states:

- (1) **Proposed:** A structural change or architectural decision suggested by an agent in a conversational turn.
- (2) **Decided:** A proposal explicitly authorized or selected by a human developer or lead agent.
- (3) **Implemented:** A decision that is mathematically verifiable via a Git diff in the codebase.
- (4) **Verified:** An implementation that successfully passes the automated test suite and type boundaries.
- (5) **Superseded / Obsolete:** A previously verified state that has been logically replaced by a newer verified implementation.

8.3 Reconciliation and Precedence

When an agent is initialized for a new task, the Context Materializer must filter the project history. If an agent previously suggested an implementation that was subsequently superseded, that proposal must be actively pruned from the injection payload. We establish the following precedence rule for conflict resolution:

Verified Implementation > Explicit Human Decision > Agent Proposal

This ensures that receiver agents never act upon hallucinated constraints or discarded hypotheses.

9 Related Work

9.1 Agent Memory and Context Storage

The challenge of agent memory has seen significant commercial and open-source development. Platforms such as Mem0 [6] and Honcho [5] provide API layers for cross-session semantic storage, allowing developers to inject historical chat context into new sessions. While effective for personal assistants, these systems generally treat all stored text equally, lacking the structural differentiation between a proposed idea and a verified implementation necessary for robust software engineering.

9.2 Codebase Knowledge Graphs

Frameworks like Cognee [4] bridge the gap between LLMs and deterministic codebases by constructing graph relationships from Abstract Syntax Trees (ASTs). The Model Context Protocol (MCP) [2] further standardizes how agents extract local filesystem states. Our approach builds upon these deterministic extraction methods but introduces a higher-order reconciliation layer that binds natural language intent to these syntactic nodes.

9.3 Multi-Agent Coordination and Governance

The Agent-to-Agent (A2A) protocol [1] represents early standardization efforts for JSON-RPC handoffs between heterogeneous agents.

However, A2A primarily addresses transport, not state governance. The conceptual foundation for our reconciliation layer draws from classical multi-agent systems literature on commitment protocols [8], adapting normative obligations to the lifecycle of LLM-generated software artifacts. The requirement for governed, authoritative state also aligns with emerging security frameworks such as SPIFFE [3] and the NIST AI RMF [7], which emphasize verifiable identity and provenance in automated systems.

10 Threats to Validity

10.1 Internal Validity

The primary threat to internal validity in our experimental design is the stochastic nature of LLM generation. To mitigate this, we pinned the execution agent (OpenCode) to a specific deterministic model configuration ([**EMPIRICAL RESULTS PENDING: Model Identifier**]) and utilized a structured JSON output format. However, inherent variance in how the model interprets the structured vs. unstructured context payloads may still influence the rework and correctness metrics.

10.2 External Validity

Our findings are currently limited to the specific tasks defined in the experimental harness ([**EMPIRICAL RESULTS PENDING: Task description**]). While the refactoring tasks are representative of typical software engineering workflows, the scalability of structured transfer payloads for highly complex, multi-repository architectural overhauls remains untested. Furthermore, our evaluation utilized specific agents (Claude Code and OpenCode); different LLM architectures may exhibit varying sensitivities to structured metadata versus raw transcripts.

10.3 Construct Validity

We measure *efficiency* primarily through task duration, token consumption, and automated test pass rates. These metrics proxy true engineering efficiency but may not capture the cognitive load required by a human supervisor to verify the agent's final implementation.

11 Discussion

[**EMPIRICAL RESULTS PENDING: This section is explicitly blocked pending empirical data from the experiments. It will discuss the implications of the findings on the design of future multi-agent software engineering platforms, highlighting trade-offs between structured governance and raw LLM context capabilities.**]

12 Conclusion

[**EMPIRICAL RESULTS PENDING: This section is explicitly blocked pending empirical data. It will summarize the transition from context amnesia to governed Project Truth, based on the findings from EXP-001 and EXP-002, and outline the necessary primitives for accountable multi-agent interoperability.**]

References

- [1] A2A Project. 2025. *A2A: Agent-to-Agent Communication Specification*. <https://github.com/a2aproject/A2A/blob/main/docs/specification.md>
- [2] Anthropic. 2025. *Model Context Protocol (MCP) Specification*. <https://modelcontextprotocol.io>
- [3] Cloud Native Computing Foundation. 2026. *SPIFFE: Secure Production Identity Framework for Everyone*. <https://spiffe.io/docs/latest/spiffe-specs/>
- [4] Cognee. 2026. *Cognee: Framework for Graph-Based AI Agent Memory*. <https://github.com/topoteretes/cognee>
- [5] Honcho AI. 2026. *Honcho: Developer Platform for State and Memory in AI Applications*. <https://honcho.dev>
- [6] Mem0. 2026. *Mem0: The Memory Layer for AI Agents*. <https://mem0.ai>
- [7] National Institute of Standards and Technology. 2023. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. Technical Report NIST AI 100-1. U.S. Department of Commerce. <https://airc.nist.gov/airmf-resources/airmf/3-sec-characteristics/>
- [8] Munindar P. Singh. 1999. An Ontology for Commitments in Multiagent Systems. *Artificial Intelligence and Law* 7, 1 (1999), 97–113. <https://www.csc2.ncsu.edu/faculty/mpsingh/papers/mas/commit.pdf>